

JDBC 连接池&DBUtils

今日内容介绍

- ◆ 使用 DBCP, C3P0 连接池完成基本数据库的操作
- ◆ 使用 DBUtils 完成 CRUD 的操作

今日内容学习目标

- ◆ 学会 C3P0 配置文件编写，以及数据源 DataSource 创建。
- ◆ 学会 DBCP 配置文件编写，以及数据源 DataSource 创建。
- ◆ 能够使用 JDBC 简化工具包 DBUtils 完成单表的增删改查操作。
- ◆ 可以用语言描述 DBUtils 底层原理

第1章 使用连接池重写工具类

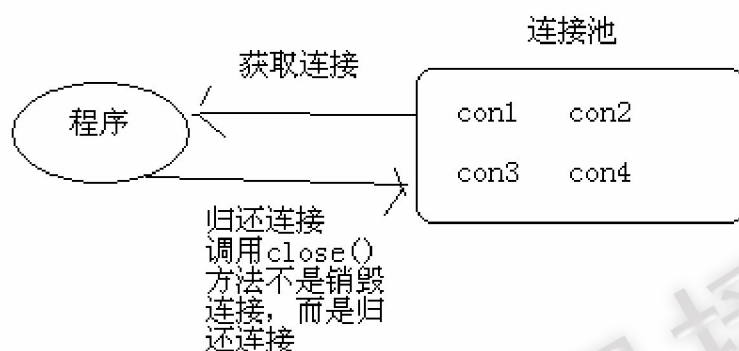
1.1 案例分析

实际开发中“获得连接”或“释放资源”是非常消耗系统资源的两个过程，为了解决此类性能问题，通常情况我们采用连接池技术，来共享连接 Connection。

1.2 连接池概述

1.2.1 概念

用池来管理 `Connection`，这样可以重复使用 `Connection`。有了池，所以我们就不用自己来创建 `Connection`，而是通过池来获取 `Connection` 对象。当使用完 `Connection` 后，调用 `Connection` 的 `close()` 方法也不会真的关闭 `Connection`，而是把 `Connection` “归还”给池。池就可以再利用这个 `Connection` 对象了。



1.2.2 规范

Java 为数据库连接池提供了公共的接口：`javax.sql.DataSource`，各个厂商需要让自己的连接池实现这个接口。这样应用程序可以方便的切换不同厂商的连接池！

常见的连接池：`DBCP`、`C3P0`。

接下来，我们就详细的学习连接池，我们将先完成自定义连接池的编写。

1.3 自定义连接池

1.3.1 案例分析

根据我们对连接池简单的理解，如果我们要编写自定义连接池，需要完成以下步骤

1. 创建连接池实现（数据源），并实现接口 `javax.sql.DataSource`。因为我们只使用该接口中 `getConnection()` 方法，简化本案例，我们将自己提供方法，而没有实现接口
2. 提供一个集合，用于存放连接，因为移除/添加操作过多，所以选择 `LinkedList`
3. 本案例在静态代码块中，为连接池初始化 3 个连接。
4. 之后程序如果需要连接，调用实现类的 `getConnection()`，本方法将从连接池（容器 `List`）获得连接。为了保证当前连接只能提供给一个线程使用，所以我们需要将连接先从连接池中移除。
5. 当用户使用完连接，释放资源时，不执行 `close()` 方法，而是将连接添加到连接池中。

1.3.2 案例实现

cn.itcast.e_pool_custom

JdbcUtils.java

TestCustomPool.java

1.3.2.1 提供容器及初始化

```
// #1 创建容器，用于存放连接 Connection
private static LinkedList<Connection> pool = new LinkedList<Connection>();

// #1.1 初始化连接池中的连接
static{
    try {
        // 1 注册驱动
        Class.forName("com.mysql.jdbc.Driver");
        for (int i = 0; i < 3; i++) {
            // 2 获得连接
            Connection conn =
                DriverManager.getConnection("jdbc:mysql://localhost:3306/day07_db", "root", "1234");
            // 3 将连接添加到连接池中
            pool.add(conn);
        }
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

1.3.2.2 获得连接

```
/**
 * #2 获得连接，从连接池中获得连接
 * @return
 */
public static Connection getConnection(){
    try {
        // 1 如果池中有连接
        if(! pool.isEmpty()){
            // 2 每一个连接 Connection，只能提供给当前一个线程使用，必须进行移除操作
            Connection conn = pool.removeFirst();
            // 3 返回刚刚获得连接
        }
    }
}
```

```
        return conn;
    }
    // 4 如果没有连接，等待 100 毫秒，然后继续
    Thread.sleep(100);
    return getConnection();
} catch (Exception e) {
    throw new RuntimeException(e);
}
}
```

1.3.2.3 归还连接

```
/**
 * #3 释放资源，当链接 connection close 时，归还给连接池
 * @param conn
 * @param st
 * @param rs
 */
public static void release(Connection conn){
    try {
        if (conn != null) {
            // conn.close(); //不是真的关闭
            pool.add(conn); //将从连接池获得连接，归还给连接池
        }
    } catch (Exception e) {
    }
}
```

1.3.2.4 测试使用

为了体现连接池优势，我们将采用多线程并发访问，使同一个连接在不同的时段，被不同的线程使用。

```
public class TestCustomPool {

    public static void main(String[] args) {
        //10 个线程，依次从连接池中获得连接
        for (int i = 0; i < 10; i++) {
            new MyThread().start();
        }
    }
}
```



```
}

class MyThread extends Thread {
    public void run() {
        try {
            //1 获得连接
            Connection conn = JdbcUtils.getConnection();

            System.out.println("使用: " + conn + " , " + Thread.currentThread());

            //2 释放资源
            JdbcUtils.release(conn);
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
};
}
```

1.4 自定义连接池：方法增强

1.4.1 需求

自定义连接池中存在严重问题，用户调用 `getConnection()` 获得连接后，必须使用 `release()` 方法进行连接的归还，如果用户调用 `conn.close()` 将连接真正的释放，连接池中将出现无连接可用。

此时我们希望，即使用户调用了 `close()` 方法，连接仍归还给连接池。`close()` 方法原有功能时释放资源，期望功能：将当前连接归还连接池。说明 `close()` 方法没有我们想要的功能，我们将对 `close()` 方法进行增强，从而实现将连接归还给连接池的功能。

1.4.2 方法增强总结

1. 继承，子类继承父类，将父类的方法进行复写，从而进行增强。
使用前提：必须有父类，且存在继承关系。
2. 装饰者设计模式，此设计模式专门用于增强方法。【】
使用前提：必须有接口
缺点：需要将接口的所有方法都实现
3. 动态代理：在运行时动态的创建代理类，完成增强操作。与装饰者相似
使用前提：必须有接口
难点：需要反射技术

4. 字节码增强，运行时创建目标类子类，从而进行增强

常见第三方框架：cglib、javassist 等

1.4.3 装饰者设计模式

设计模式：专门为解决某一类问题，而编写的固定格式的代码。

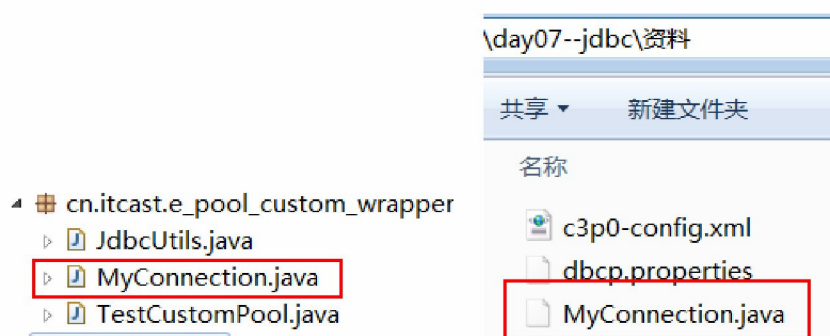
装饰者固定结构：接口 A，已知实现类 C，需要装饰者创建代理类 B

1. 创建类 B，并实现接口 A
2. 提供类 B 的构造方法，参数类型为 A，用于接收 A 接口的其他实现类（C）
3. 给类 B 添加类型为 A 成员变量，用于存放 A 接口的其他实现类
4. 增强需要的方法
5. 实现不需要增强的方法，方法体内调用成员变量存放的其他实现类对应的方法

```
A a = ...C;
B b = new B(a);
class B implements A{
    private A a;
    public B(A a){
        this.a = a;
    }
    //增强的方法
    public void close(){
    }
    //不需要增强的方法
    public void commit(){
        this.a.commit();
    }
}
```

1.4.4 实现

1.4.4.1 装饰类



```
public class MyConnection implements Connection {
    private Connection conn;
    private List<Connection> pool;

    public MyConnection(Connection conn){
        this.conn = conn;
    }

    /* 因为与自定义连接池有关系，所以需要另外添加一个构造方法 */
    public MyConnection(Connection conn,List<Connection> pool){
        this.conn = conn;
        this.pool = pool;
    }

    /* 以下是增强的方法 */

    @Override
    public void close() throws SQLException {
        //将调用当前 close 的链接 Connection 对象添加到链接池中
        // System.out.println("连接归还: " + this);
        this.pool.add(this);
    }

    /* 以下是不需要增强的方法 */

    @Override
    public void commit() throws SQLException {
        this.conn.commit();
    }
    ....
}
```

1.4.4.2 使用装饰类（包装类）

将由 DriverManager 创建的连接，使用装饰类包装一下，然后添加到连接池中，构造方法中将容器 pool 传递进去，方便连接的归还。

```
MyConnection.java  JdbcUtils.java  TestCustomPool.java
15      //1 注册驱动
16      Class.forName("com.mysql.jdbc.Driver");
17      for (int i = 0; i < 3; i++) {
18          //2 获得连接
19          Connection conn = DriverManager.getConnection("jdbc:mysql://lc
20          //3 将连接添加到连接池中
21          // 添加的连接被装饰者类包装过的
22          MyConnection myConn = new MyConnection(conn,pool);
23          pool.add(myConn);
24      }
25  } catch (Exception e) {          之前的代码: pool.add(conn)
26      throw new RuntimeException(e);
27  }
28  }
```

1.4.4.3 使用连接

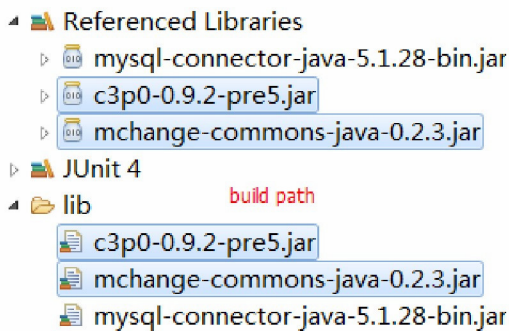
```
MyConnection.java  JdbcUtils.java  TestCustomPool.java
17  public void run() {
18      try {
19          //1 获得连接
20          Connection conn = JdbcUtils.getConnection();
21
22          System.out.println("使用: " + conn + " , " + Thread.currentThread());
23
24          //2释放资源
25          //JdbcUtils.release(conn);
26          conn.close();          调用的close()其实是MyConnection的close()方法，该方法内部将当前连接归还到连接池
27      } catch (Exception e) {
28          throw new RuntimeException(e);
29      }
30  }
```

1.5 C3P0 连接池

C3P0 开源免费的连接池！目前使用它的开源项目有：Spring、Hibernate 等。使用第三方工具需要导入 jar 包，c3p0 使用时还需要添加配置文件 c3p0-config.xml

1.5.1 导入 jar 包

我们使用的 0.9.2 版本，需要导入 2 个 jar 包



1.5.2 配置文件

- 配置文件名称: c3p0-config.xml (固定)
- 配置文件位置: src (类路径)
- 配置文件内容: 命名配置

```
<c3p0-config>
    <!-- 命名的配置 -->
    <named-config name="itcast">
        <!-- 连接数据库的 4 项基本参数 -->
        <property name="driverClass">com.mysql.jdbc.Driver</property>
        <property name="jdbcUrl">jdbc:mysql://127.0.0.1:3306/day07_db</property>
        <property name="user">root</property>
        <property name="password">1234</property>
        <!-- 如果池中数据连接不够时一次增长多少个 -->
        <property name="acquireIncrement">5</property>
        <!-- 初始化连接数 -->
        <property name="initialPoolSize">20</property>
        <!-- 最小连接数 -->
        <property name="minPoolSize">10</property>
        <!-- 最大连接数 -->
        <property name="maxPoolSize">40</property>
        <!-- -JDBC 的标准参数,用以控制数据源内加载的 PreparedStatements 数量 -->
        <property name="maxStatements">0</property>
        <!-- 连接池内单个连接所拥有的最大缓存 statements 数 -->
        <property name="maxStatementsPerConnection">5</property>
    </named-config>
</c3p0-config>
```

- 配置文件内容: 默认配置

```
<c3p0-config>
    <!-- 默认配置,如果没有指定则使用这个配置 -->
    <default-config>
        <property name="driverClass">com.mysql.jdbc.Driver</property>
        <property name="jdbcUrl">jdbc:mysql://127.0.0.1:3306/day07_db</property>
        <property name="user">root</property>
```



```

<property name="password">1234</property>
<property name="checkoutTimeout">30000</property>
<property name="idleConnectionTestPeriod">30</property>
<property name="initialPoolSize">10</property>
<property name="maxIdleTime">30</property>
<property name="maxPoolSize">100</property>
<property name="minPoolSize">10</property>
<property name="maxStatements">200</property>
<user-overrides user="test-user">
    <property name="maxPoolSize">10</property>
    <property name="minPoolSize">1</property>
    <property name="maxStatements">0</property>
</user-overrides>
</default-config>

```

1.5.3 常见配置项

分类	属性	描述
必须项	user	用户名
	password	密码
	driverClass	驱动 mysql 驱动, com.mysql.jdbc.Driver
	jdbcUrl	路径 mysql 路径, jdbc:mysql://localhost:3306/数据库
基本配置	acquireIncrement	连接池无空闲连接可用时, 一次性创建的新连接数 默认值: 3
	initialPoolSize	连接池初始化时创建的连接数 默认值: 3
	maxPoolSize	连接池中拥有的最大连接数 默认值: 15
	minPoolSize	连接池保持的最小连接数。
	maxIdleTime	连接的最大空闲时间。如果超过这个时间, 某个数据库连接还没有被使用, 则会断开掉这个连接, 如果为 0, 则永远不会断开连接。 默认值: 0
管理连接池的大小和连接的生存时间 (扩展)	maxConnectionAge	配置连接的生存时间, 超过这个时间的连接将由连接池自动断开丢弃掉。当然正在使用的连接不会马上断开, 而是等待它 close 再断开。配置为 0 的时候则不会对连接的生存时间进行限制。默认值 0
	maxIdleTimeExcessConnections	这个配置主要是为了减轻连接池的负载, 配置

		不为 0，则会将连接池中的连接数量保持到 minPoolSize，为 0 则不处理。
配置 Prepared Statement 缓存(扩展)	maxStatements	连接池为数据源缓存的 PreparedStatement 的总数。由于 PreparedStatement 属于单个 Connection,所以这个数量应该根据应用中平均连接数乘以每个连接的平均 PreparedStatement 来计算。为 0 的时候不缓存，同时 maxStatementsPerConnection 的配置无效。
	maxStatementsPerConnection	连接池为数据源单个 Connection 缓存的 PreparedStatement 数，这个配置比 maxStatements 更有意义，因为它缓存的服务对象是单个数据连接，如果设置的好，肯定是可以提高性能的。为 0 的时候不缓存。

1.5.4 编写工具类

C3P0 提供核心工具类：ComboPooledDataSource，如果要使用连接池，必须创建该类的实例对象。

- new ComboPooledDataSource("名称"); 使用配置文件“命名配置”

- ◆ <named-config name="itcast">

- new ComboPooledDataSource(); 使用配置文件“默认配置”

- ◆ <default-config>

- cn.itcast.f_pool_c3p0

- C3P0Utils.java

- TestC3P0.java

- c3p0-config.xml

```
public class C3P0Utils{

    //使用默认配置
    // private static ComboPooledDataSource dataSource = new ComboPooledDataSource();
    //使用命名配置
    private static ComboPooledDataSource dataSource = new
ComboPooledDataSource("itcast");

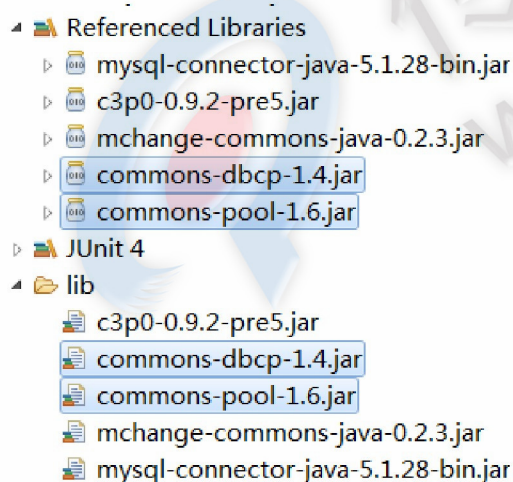
    /**
     * 获得数据源（连接池）
     * @return
     */
    public static DataSource getDataSource(){
        return dataSource;
    }
}
```

```
/**
 * 获得连接
 * @return
 * @throws SQLException
 */
public static Connection getConnection(){
    try {
        return dataSource.getConnection();
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

1.6 DBCP 连接池

DBCP 也是一个开源的连接池，是 Apache Common 成员之一，在企业开发中也比较常见，tomcat 内置的连接池。

1.6.1 导入 jar 包



1.6.2 配置文件

- 配置文件名称: *.properties
- 配置文件位置: 任意，建议 src (classpath/类路径)
- 配置文件内容: properties 不能编写中文，不支持在 STS 中修改，必须使用记事本修改内容，否则中文注释就乱码了

```
#连接设置
driverClassName=com.mysql.jdbc.Driver
url=jdbc:mysql://localhost:3306/day07_db
username=root
password=1234

#<!-- 初始化连接 -->
initialSize=10

#最大连接数量
maxActive=50

#<!-- 最大空闲连接 -->
maxIdle=20

#<!-- 最小空闲连接 -->
minIdle=5

#<!-- 超时等待时间以毫秒为单位 6000 毫秒/1000 等于 60 秒 -->
maxWait=60000

#JDBC 驱动建立连接时附带的连接属性属性的格式必须为这样: [属性名=property;]
#注意: "user" 与 "password" 两个属性会被明确地传递, 因此这里不需要包含他们。
connectionProperties=useUnicode=true;characterEncoding=gbk

#指定由连接池所创建的连接的自动提交 (auto-commit) 状态。
defaultAutoCommit=true

#driver default 指定由连接池所创建的连接的只读 (read-only) 状态。
#如果没有设置该值, 则 "setReadOnly" 方法将不被调用。(某些驱动并不支持只读模式, 如: Informix)
defaultReadOnly=

#driver default 指定由连接池所创建的连接的事务级别 (TransactionIsolation)。
#可用值为下列之一: (详情可见 javadoc。) NONE, READ_UNCOMMITTED, READ_COMMITTED,
REPEATABLE_READ, SERIALIZABLE
defaultTransactionIsolation=READ_UNCOMMITTED
```

1.6.3 常见配置项

分类	属性	描述
必须项	driverClassName	

	url	
	username	
	password	
基本项	maxActive	最大连接数量
	minIdle	最小空闲连接
	maxIdle	最大空闲连接
	initialSize	初始化连接
优化配置（扩展）	logAbandoned	连接被泄露时是否打印
	removeAbandoned	是否自动回收超时连接
	removeAbandonedTimeout	超时时间(以秒数为单位)
	maxWait	超时等待时间以毫秒为单位 1000 等于 60 秒
	timeBetweenEvictionRunsMillis	在空闲连接回收器线程运行期间休眠的时间值,以毫秒为单位
	numTestsPerEvictionRun	在每次空闲连接回收器线程(如果有)运行时检查的连接数量
	minEvictableIdleTimeMillis	连接在池中保持空闲而不被空闲连接回收器线程

参考文档: <http://commons.apache.org/proper/commons-dbcp/configuration.html>

1.6.4 编写工具类

```

public class DBCPUtils{

    private static DataSource dataSource;

    static{
        try {
            //1 加载配置文件，获得文件流
            InputStream is =
DBCPUUtils.class.getClassLoader().getResourceAsStream("dbcp.properties");
            //2 使用 Properties 处理配置文件
            Properties props = new Properties();
            props.load(is);
            //3 使用工具类创建连接池（数据源）
            dataSource = BasicDataSourceFactory.createDataSource(props);
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
}

```

```
public static DataSource getDataSource() {  
    return dataSource;  
}  
  
public static Connection getConnection() {  
    try {  
        return dataSource.getConnection();  
    } catch (Exception e) {  
        throw new RuntimeException(e);  
    }  
}  
}
```

第2章 使用DBUtils 增删改查的操作

2.1 案例分析

如果只使用 JDBC 进行开发，我们会发现冗余代码过多，为了简化 JDBC 开发，本案例我们讲采用 apache commons 组件一个成员：DBUtils。

DBUtils 就是 JDBC 的简化开发工具包。需要使用技术：连接池（获得连接），SQL 语句都没有少。

2.2 案例相关知识

2.2.1 JavaBean 组件

JavaBean 就是一个类，在开发中常用语封装数据。具有如下特性

1. 需要实现接口：java.io.Serializable ，通常偷懒省略了。
2. 提供私有字段：private 类型 字段名；
3. 提供 getter/setter 方法：
4. 提供无参构造

```
public class Category {  
  
    private String cid;  
    private String cname;  
  
    public Category() {  
        super();  
    }  
}
```



```
    }  
    public String getCid() {  
        return cid;  
    }  
    public void setCid(String cid) {  
        this.cid = cid;  
    }  
    public String getName() {  
        return cname;  
    }  
    public void setName(String cname) {  
        this.cname = cname;  
    }  
    ...toString...  
}
```

2.3 DBUtils 完成 CRUD

2.3.1 概述

DBUtils 是 java 编程中的数据库操作实用工具，小巧简单实用。

DBUtils 封装了对 JDBC 的操作，简化了 JDBC 操作，可以少写代码。

Dbutils 三个核心功能介绍

- QueryRunner 中提供对 sql 语句操作的 API.
- ResultSetHandler 接口，用于定义 select 操作后，怎样封装结果集.
- DbUtils 类，它就是一个工具类，定义了关闭资源与事务处理的方法

2.3.2 QueryRunner 核心类

- QueryRunner(DataSource ds), 提供数据源（连接池），DBUtils 底层自动维护连接 connection
- update(String sql, Object... params) ， 执行更新数据
- query(String sql, ResultSetHandler<T> rsh, Object... params) ， 执行查询

2.3.3 ResultSetHandler 结果集处理类

ArrayHandler	将结果集中的第一条记录封装到一个 Object[] 数组中，数组中的每一个元素就是这条记录中的每一个字段的值
--------------	--

ArrayListHandler	将结果集中的每一条记录都封装到一个 Object[]数组中, 将这些数组在封装到 List 集合中。
BeanHandler	将结果集中第一条记录封装到一个指定的 javaBean 中。
BeanListHandler	将结果集中每一条记录封装到指定的 javaBean 中, 将这些 javaBean 在封装到 List 集合中
ColumnListHandler	将结果集中指定的列的字段值, 封装到一个 List 集合中
KeyedHandler	将结果集中每一条记录封装到 Map<String,Object>, 在将这个 map 集合做为另一个 Map 的 value, 另一个 Map 集合的 key 是指定的字段的值。
MapHandler	将结果集中第一条记录封装到了 Map<String,Object>集合中, key 就是字段名称, value 就是字段值
MapListHandler	将结果集中每一条记录封装到了 Map<String,Object>集合中, key 就是字段名称, value 就是字段值, 在将这些 Map 封装到 List 集合中。
ScalarHandler	它是用于单数据。例如 select count(*) from 表操作。

2.3.4 DbUtils 工具类

closeQuietly(Connection conn) 关闭连接, 如果有异常 try 后不抛。

commitAndCloseQuietly(Connection conn) 提交并关闭连接

rollbackAndCloseQuietly(Connection conn) 回滚并关闭连接

2.3.5 实现

2.3.5.1 添加

```
@Test
public void demo01() {
    try {

        //1 核心类
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());
        //2 sql
        String sql = "insert into category(cid,cname) values('c001','家电')";
        //3 参数
        Object[] params = {"c006","家电"};
        //4 执行
        queryRunner.update(sql, params);

    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

```
}  
}
```

2.3.5.2 更新

```
@Test  
public void demo02() throws Exception{  
    try {  
  
        //1 核心类  
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());  
        //2 sql  
        String sql = "update category set cname=? where cid = ?";  
        //3 参数  
        Object[] params = {"家电", "c006"};  
        //4 执行  
        queryRunner.update(sql, params);  
  
    } catch (Exception e) {  
        throw new RuntimeException(e);  
    }  
  
}
```

2.3.5.3 删除

```
@Test  
public void demo03() throws Exception{  
    try {  
  
        //1 核心类  
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());  
        //2 sql  
        String sql = "delete from category where cid = ?";  
        //3 参数  
        Object[] params = {"c006"};  
        //4 执行  
        queryRunner.update(sql, params);  
  
    } catch (Exception e) {  
        throw new RuntimeException(e);  
    }  
  
}
```

```
    }  
}
```

2.3.5.4 通过 id 查询

```
@Test  
public void demo04() throws Exception{  
  
    try {  
  
        //1 核心类  
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());  
        //2 sql  
        String sql = "select * from category where cid=?";  
        //3 参数  
        Object[] params = {"c003"};  
        //4 执行  
        Category category = queryRunner.query(sql, new  
        BeanHandler<Category>(Category.class), params);  
  
        System.out.println(category);  
    } catch (Exception e) {  
        throw new RuntimeException(e);  
    }  
}
```

2.3.5.5 查询所有

```
@Test  
public void demo05() throws Exception{  
  
    try {  
  
        //1 核心类  
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());  
        //2 sql  
        String sql = "select * from category";  
        //3 参数  
        Object[] params = {};  
        //4 执行  
        List<Category> allCategory = queryRunner.query(sql, new
```

```
BeanListHandler<Category>(Category.class), params);

        for (Category category : allCategory) {
            System.out.println(category);
        }

    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

2.3.5.6 总记录数

```
@Test
public void demo06() throws Exception{

    try {

        //1 核心类
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());
        //2 sql
        String sql = "select count(*) from category";
        //3 参数
        Object[] params = {};
        //4 执行
        Long numLong = (Long) queryRunner.query(sql, new ScalarHandler(), params);

        int num = numLong.intValue();

        System.out.println(num);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

第3章 总结

